A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline

Dung Van Bui
University of Information Technology
VNU-HCM
Ho Chi Minh City, Vietnam
dungbv.17@grad.uit.edu.vn

Thai Vu Nguyen
University of Transport and Communications
Ho Chi Minh City Campus
Ho Chi Minh City, Vietnam
nvthai@utc2.edu.vn

Nhut Tri Do
University of Information Technology
VNU-HCM
Ho Chi Minh City, Vietnam
trinhutdo@uit.edu.vn

Van Du Nguyen
Faculty of Information Technology
Nong Lam University
Ho Chi Minh City, Vietnam
nvdu@hcmuaf.edu.vn

***Abstract*—Machine-learning-based network intrusion detection systems (IDS) typically process hundreds of network-flow features, which is computationally expensive and hard to deploy at the edge. This paper presents a systematic comparative study of four dimensionality-reduction strategies on Apache Spark ML: a baseline using all features, selection by Random Forest Importance, selection by SHAP, and PCA. Each strategy is evaluated across eight classification algorithms together with two ensemble methods on the CICIDS2017 dataset [1]. Unlike most prior work, we emphasize the *rigor of the evaluation pipeline*: (i) we remove the label-leaking feature `destination_port` and deduplicate flows at the feature level before splitting the data; (ii) we fix the comparison configuration *a priori* to avoid selection bias; (iii) we select models on a validation set held out from the training set, together with an in-distribution stability check and a *cross-dataset* generalization evaluation (CICIDS2017 ↔ CSE-CIC-IDS2018); (iv) we assess statistical significance using a paired permutation test over multiple data resamplings, together with Nadeau–Bengio corrected confidence intervals and effect sizes; and (v) we add a per-attack-type multiclass evaluation. The study aims to verify whether feature selection retains most of the classification performance while cutting the number of features by about 60%, and whether SHAP Top-30 balances accuracy, explainability, and cost so as to be suitable for export to the Jetson Orin Nano Super edge device (see the related work); the quantitative results are reported in the experimental section.**

***Index Terms*—Intrusion detection, Apache Spark, Feature selection, SHAP, Data leakage**

I. Introduction

Cyberattacks are growing in scale and sophistication, demanding IDS that are both accurate and scalable on large data [2], [3]. Machine learning has become the dominant approach for anomaly-based IDS, but modern network-flow datasets such as CICIDS2017 [1] provide nearly 80 features per flow, which increases training, inference, and memory cost — especially critical for edge deployment. Dimensionality reduction and feature selection are therefore key steps. However, different strategies such as embedded, unsupervised, or explanation-based ones measure "importance" in non-equivalent ways, and they are rarely compared under the same distributed pipeline.

A less-discussed but equally serious issue is the *trustworthiness of the evaluation pipeline*. Many reports on CICIDS2017 achieve near-perfect binary F1; this is usually a sign of data leakage rather than of model capability. *Data leakage* is the situation in which information that should not be available at real deployment time enters the training/evaluation process, inflating scores so that they no longer reflect true generalization. Labeling errors and duplicate flows in CICIDS2017 itself have been documented and shown to affect evaluation results substantially [4]. There are two common sources of leakage. First, the `destination_port` feature directly encodes the attacked service, because CICIDS2017 generates attack scenarios aimed at fixed ports, so it becomes a "shortcut" for the model to memorize the port → label mapping. Second, feature-level duplicate flows are not removed by full-duplicate removal, so a random split places the same sample into both the training and test sets. This paper handles both before comparing methods.

Main contributions. (1) A uniform comparison of RF Importance, SHAP, and PCA over the same eight Spark ML algorithms and two ensembles; (2) Integration of SHAP-based explanation [5] with a direct contrast against RF Importance [6]; (3) A *leakage-aware and statistically tested* evaluation pipeline: removing the leaking feature, deterministic feature-level deduplication (label-conflicting groups are removed entirely), model selection on a validation set held out from the training set, and a paired permutation test (exact enumeration) over multiple resamplings with Nadeau–Bengio correction and effect size; (4) A per-attack-type multiclass evaluation, clarifying where the methods truly differ; (5) An *edge-aware* model-selection recommendation: jointly considering F1, the number of features, and model size for the Jetson Orin Nano Super.

Section II presents related work; Section III describes the method and evaluation pipeline; Section IV reports the experiments; Section V discusses; Section VI states threats to validity; and Section VII concludes.

II. Related Work

IDS surveys [2], [3], [7] show that both traditional machine learning and deep learning are widely applied on standard benchmarks such as CICIDS2017 [1]. Many works focus on a single algorithm, for instance online deep learning [8] or deep networks [9], without a uniform comparison of multiple dimensionality-reduction strategies on the same distributed pipeline. Apache Spark ML [10], [11] supports large-scale processing, but few studies combine feature selection, model explanation, and statistical testing in a single unified experimental framework. Recent edge IDS studies deploy deep-learning models on NVIDIA Jetson [12], [13], but they typically focus on a single model and on accuracy, and rarely compare multiple dimensionality-reduction strategies under a leakage-aware evaluation pipeline.

Feature selection. Variable-selection theory [14] distinguishes three groups: filter, wrapper, and embedded. Random Forest Importance [6] (embedded) and PCA [15] (unsupervised) are two popular directions for IDS but measure different quantities. SHAP [5] provides consistent local explanations and has been used for IDS [16], but is rarely contrasted directly with RF Importance on the same Spark training set.

Concept drift, anomaly, and evaluation. Concept drift [17] affects IDS stability over time; anomaly-detection studies [18]–[21] typically separate the filter from the classifier, and autoencoders are also used to reduce feature dimensionality for IDS on edge devices [21]. However, data leakage and statistical-significance testing in method comparison are rarely handled explicitly on CICIDS2017. This paper fills that gap. Table I summarizes the comparison.

III. Method

A. Overview of the Spark ML pipeline

The offline pipeline consists of sequential steps: data cleaning, deduplication, `VectorAssembler`, `StandardScaler`, an (optional) dimensionality-reduction transform, and then the classifier. Binary labels: Benign (0) and Attack (1). The entire `fit` of StandardScaler/PCA and the feature ranking use *only* the training set; the test set serves as the final holdout.

B. Leakage-aware data preparation

We merge the eight CICIDS2017 CSV files, normalize column names, replace infinite/missing values, and then apply two leakage-removal steps *before* splitting the data:

- **Removing the label-leaking feature.** `destination_port` (and `source_port` if present) is removed because it directly encodes the target service of the attack; keeping it would let the model memorize the port $\rightarrow$ label mapping instead of learning flow behavior. An ablation (keep vs. remove the port) quantifies the impact.
- **Feature-level deduplication (deterministic).** Beyond removing fully duplicate records, we remove flows that are *identical across all feature columns* before splitting, preventing the same sample from appearing in both the training and test sets. For groups of flows that are feature-identical but have *conflicting labels* (Benign and Attack — a known error in CICIDS2017 [4]), the entire group is removed rather than arbitrarily keeping one row depending on data partitioning; the numbers of removed groups and rows are reported (270 groups, 44,260 rows), ensuring the deduplication result is deterministic and reproducible.

After processing, the data are split into **80% training / 20% test** (with a fixed seed), stored in Parquet format, and reused throughout the experiments. The number of remaining numeric features is 77, and the number of samples after deduplication is 1,785,349/447,275 ($\approx$2,232,624 total after feature-level dedup).

C. Classifiers and the four dimensionality-reduction strategies

Eight algorithms: Decision Tree, Logistic Regression, SVM (LinearSVC), Naive Bayes, Random Forest, GBT, XGBoost, MLP; together with two ensembles (weighted Hybrid Bagging Top-3 and Soft Voting). LightGBM is excluded because its native library (via SynapseML) supports only x86_64 and cannot run on the Jetson Orin Nano Super edge device (ARM64), which is the deployment target of this study; therefore a LightGBM model cannot be included in the pipeline. Within the gradient-boosting family, XGBoost and GBT act as representatives. The four strategies: **Baseline** (all features); **RF Top-K** ($K \in \{20, 30, 40\}$); **SHAP Top-K**; **PCA** ($k \in \{20, 30, 40\}$). The RF and SHAP rankings use *only* the training set; SHAP is computed on a *class-stratified* sample so that both benign and attack traffic are represented. PCA is fixed at k=40, more than the Top-30 of the selection methods, because PCA is an *extraction*: each principal component is a linear combination carrying only part of the variance, so more components are needed to retain an equivalent amount of variance. Within the range $\{20, 30, 40\}$, k=40 retains the most variance and is therefore the fairest representative point for PCA.

D. Hyperparameters and class-imbalance handling

The hyperparameters (Table II) are *fixed a priori* and shared across all dimensionality-reduction methods, so that the observed differences correctly reflect the *feature-selection method* rather than per-model tuning; hyperparameter tuning is separated into a distinct stage (grid search with cross-validation) and applied only to the selected configuration.

Table I
COMPARISON WITH RELATED IDS STUDIES ON CICIDS2017 AND SPARK/EDGE.

Work	**Focus**	**Data**	**Dim. reduction / XAI**	**Novelty of this work**
Khraisat et al. [3]	IDS survey	Multiple	General	Uniform empirical comparison of 4 strategies under the same pipeline
Vinayakumar et al. [9]	Deep learning for IDS	CICIDS2017	None	8 Spark ML algorithms + 2 ensembles; explicit data-leakage control
Ferrag et al. [7]	Deep learning, multi-dataset comparison	Multiple	None	Compares RF/SHAP/PCA under a leakage-aware pipeline + paired statistical testing
Alrawashdeh & Purdy [8]	Online deep learning	NSL-KDD	None	Leakage-controlled evaluation; SHAP model explanation
Sarhan et al. [16]	XAI for IDS	NetFlow (NF-*)	SHAP	SHAP on Spark ML; direct comparison with RF Importance and PCA on CICIDS2017
Baalaji et al. [12]	Deep-learning ensemble on IoT-edge	IoT-edge	None	Compares several dimensionality-reduction strategies rather than a single model; statistical testing
Asghar et al. [13]	Explainable edge IDS	Multiple	Metaheuristic	Leakage-aware pipeline + cross-dataset evaluation 2017↔2018
Hasan et al. [21]	AE feature learning for IIoT-edge	IIoT	Autoencoder	Supervised feature selection (RF/SHAP) vs. extraction; model selection on validation
This paper	**Feature selection + Spark ML**	**CICIDS2017, CSE-CIC-IDS2018**	**RF/SHAP/PCA**	**Unified pipeline: leakage removal, selection on validation, exact permutation + Nadeau–Bengio, multiclass, cross-dataset**

The stochastic models (RF, GBT, XGBoost, MLP) fix the `seed` for reproducibility. Class imbalance is handled *uniformly*. Every model that supports `weightCol` — namely Decision Tree, Logistic Regression, SVM, Naive Bayes, Random Forest, and GBT — is assigned class weights according to the benign/attack ratio, equivalent to XGBoost's `scale_pos_weight` mechanism. MLP alone does not support sample weights in Spark ML and is therefore trained without weights, as noted in the limitations.

E. Two-stage comparison design

To avoid selection bias from trying many configurations on the same test set, we separate the *exploration stage* (sweeping K, fully reported per K) from the *confirmation stage* (four configurations fixed a priori: Baseline, RF Top-30, SHAP Top-30, PCA k=40). The comparative conclusions rest on the four fixed configurations; we do not pick the configuration with the highest F1 after observing the results on the test set. Within each configuration, the *representative model* and the *method pair* entered into statistical testing are also chosen by F1 on a validation set (20% held out from the training set, fixed seed) — the test set does not participate in any selection step.

F. Rigorous evaluation pipeline

Metrics. Accuracy, Precision, Recall, F1, AUC-ROC, AUC-PR; training/prediction time; model size. Because the data are imbalanced, AUC-PR and attack-class recall are prioritized over accuracy.

In-distribution stability check. The representative model of each method is re-evaluated on a subset of the test set (disjoint from the training set). To be clear: this is only an *in-distribution stability* check, not a distribution-shift test; the true generalization assessment lies in the cross-dataset experiment (Section IV).

Statistical-significance testing. The two top methods are trained on N independent *resamplings* of the training/test split (default N=6), and in each round both use the *same* split so as to be paired. As a result, the variance reflects the effect of data sampling rather than only of model initialization. The 95% confidence interval is computed using the t distribution ($t_{0.975,N-1}$, not the value 1.96); because the resamplings overlap in training data, we report *in parallel* the Nadeau–Bengio corrected confidence interval, with the variance multiplied by $(1/N + n_{\text{test}}/n_{\text{train}})$, and take the corrected interval as the basis for conclusions. Significance is tested by a *two-sided paired permutation test* on the per-round F1 difference, *exactly enumerating* all 2^N sign permutations (with N=6, that is 64 permutations) rather than Monte Carlo sampling; because the test is two-sided, the smallest attainable p is $2/2^N$, so $N \geq 6$ is needed to be able to reach below 0.05 (in contrast to a single-fixed-split / three-seed design, which cannot get past $p \approx 0.125$). Together with p, we report the paired Cohen's d

Table II
MAIN HYPERPARAMETERS OF THE EIGHT CLASSIFIERS (FIXED A PRIORI).

Model	Main hyperparameters
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8, colsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01
Class balancing: `weightCol` (6 models) / `scale_pos_weight` (XGBoost); MLP unweighted.	

effect size: with small N, a non-significant p-value is by itself weak evidence for "equivalence".

Multiclass evaluation. Because binary classification on CICIDS2017 is almost perfectly separable, we add a *per-attack-type multiclass* evaluation, comprising per-class precision/recall/F1, macro-F1, and a confusion matrix. It is here that rare classes such as Heartbleed, Infiltration, Web Attack, and Bot reveal the real differences among methods.

IV. EXPERIMENTS AND RESULTS

A. Setup

CICIDS2017; an 80/20 training/test split after leakage removal (Section III-B); Apache Spark 3.4.1, Java 17; hardware: a cluster of 2× Jetson Orin Nano Super (8 GB) and a coordinator host. Every script logs the run configuration (seed, environment variables, library versions) to ensure reproducibility. The full source code, configuration, and feature lists are publicly available at https://github.com/vuthainguyen1602/Thesis_IDS (branch `develop`); the pipeline can be reproduced on a single machine and does not require an edge-device cluster.

B. Port-feature ablation

Table III and Figure 1 quantify the impact of `destination_port`. After feature-level deduplication, the port's isolated effect is *small*: binary F1 is 0.9965 with the port and 0.9955 without it (a ≈0.001 gap) — the in-domain binary task is near saturation and the dominant leakage source (duplicate flows) is already handled; the direction (keeping the port yields slightly higher F1/recall) is nonetheless consistent with leakage. All subsequent results use the *port-removed* configuration.

Table III
IMPACT OF THE LEAKING FEATURE `DESTINATION_PORT` (RANDOM FOREST FIXED A PRIORI PER THE TABLE II CONFIGURATION, CLASS-WEIGHTED; BINARY).

Configuration	F1	Recall (Attack)	AUC-PR
Keep port	0.9965	0.9988	0.9999
Remove port (proposed)	0.9955	0.9961	0.9997

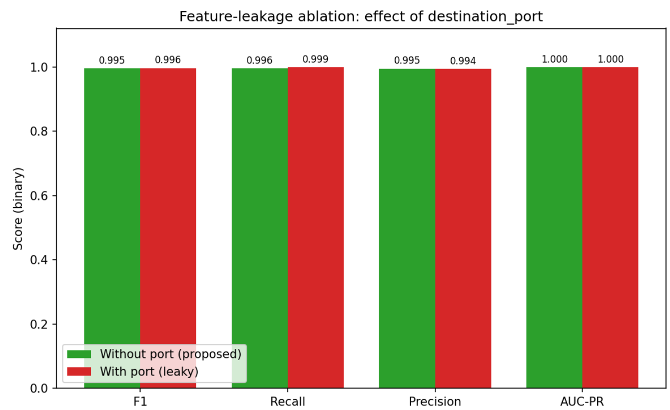

Figure 1. Impact of the leaking feature `destination_port` (Random Forest, binary).

C. Comparison of the four dimensionality-reduction methods

Table IV summarizes the best model per method, and Figure 2 visualizes F1 for each method × algorithm pair. The comparison design aims to verify whether feature selection (RF/SHAP Top-30) retains near-baseline performance while cutting ≈60% of the features; PCA k=40 reduces dimensionality in an unsupervised manner at the cost of explainability. All quantitative values are taken directly from the released pipeline output (`cross_method_summary.csv`).

D. Statistical testing (paired resampling)

Table V reports the mean F1 ± 95% CI (t distribution) over N=6 resamplings for the two top methods, together with the paired permutation-test p-value. Conclusion: the two methods differ *significantly only at the floor* ($p \approx 0.031$, Cohen's $d \approx 1.94$), yet the absolute F1 gap is tiny (≈0.0001), so they are *practically equivalent*.

E. Robustness and multiclass evaluation

In the in-distribution stability check (a subset of the test set), the ranking of the methods *is preserved* (XGBoost leads in all four methods); details in `robustness_holdout_summary.csv`. The multiclass evaluation (Figure 4, Table VI) reveals a sharp contrast between weighted-F1 (≈0.998) and macro-F1 (≈0.806): the majority

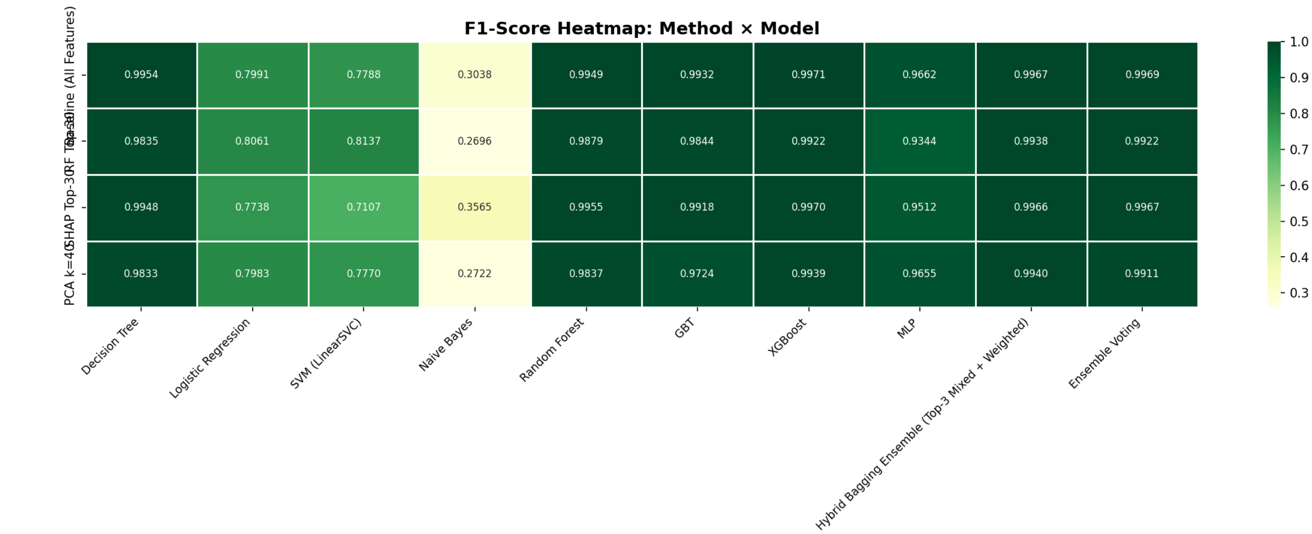

Figure 2. F1 heatmap by method and algorithm (binary, port removed).

Table IV
SUMMARY OF THE BEST MODEL PER METHOD (VALUES FROM `CROSS_METHOD_SUMMARY.CSV`).

Method	Model	F1	AUC-PR	Train (s)	Size (MB)
Baseline	XGBoost	0.9971	0.9999	4615.5	2.32
RF Top-30	XGBoost	0.9922	0.9998	1295.1	2.52
SHAP Top-30	XGBoost	0.9970	0.9999	1226.1	0.003
PCA k=40	XGBoost	0.9939	0.9999	1444.8	3.82

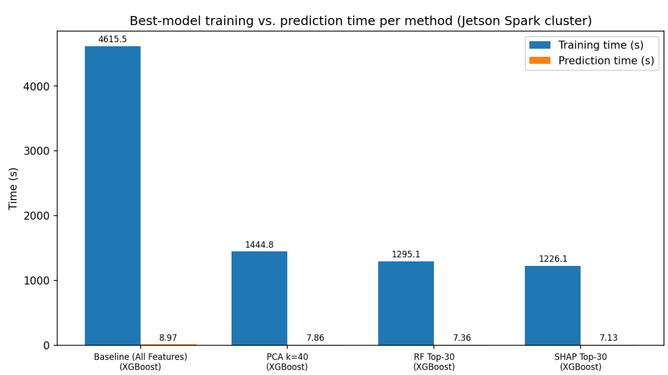

Figure 3. Training and inference time of the representative model of each method, measured on the Spark cluster used for training (Section IV); inference latency/throughput measurements on the edge device belong to a companion systems paper (under submission), not this paper.

Table V
MULTI-RESAMPLING STABILITY AND PAIRED TESTING (VALUES FROM `STATISTICAL_VALIDITY_MULTISEED.CSV`).

Method	Model	F1 (mean ± CI95)	p (paired)
Baseline	XGBoost	0.99734 ± 0.00007	0.031
SHAP Top-30	XGBoost	0.99720 ± 0.00009	

classes achieve high F1, while rare classes such as Web Attack (XSS, SQL Injection) and Bot have very low recall (0.03–0.46) — where feature selection makes the clearest difference.

Table VI
PER-CLASS PRECISION/RECALL/F1 (EXTRACTED FROM `PER_CLASS_METRICS.CSV`).

Class	Support	Precision	Recall	F1
Benign	380,119	0.9981	0.9997	0.9989
Web Attack – XSS	111	1.0000	0.0270	0.0526
13 remaining classes — see `per_class_metrics.csv`				
macro-F1 = 0.806 weighted-F1 = 0.998				

F. *Cross-dataset generalization*

To go beyond *in-domain* evaluation, we train on one dataset and test on a *different* one (*cross-dataset* evaluation; and vice versa) on the *shared* leakage-free feature set of the two datasets. The gap between in-domain F1 (A→A) and cross-dataset F1 (A→B) in Table VII quantifies the *generalization* ability of the leakage-free model. This is a genuine distribution-shift test, unlike the in-distribution holdout above. Because CSE-CIC-IDS2018 is very large (∼16 million flows), we use a *label-stratified* sample (fraction *[xd_frac]*, ∼*[xd_n2018]* flows, `seed=42`) that preserves the class proportions; CICIDS2017 uses ∼*[xd_n2017]* flows. The sample sizes are reported explicitly to ensure reproducibility.

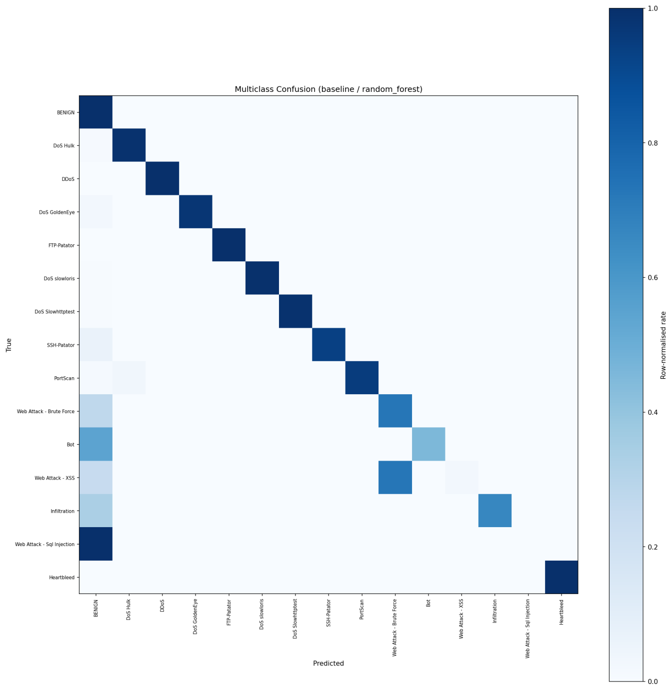

Figure 4. Multiclass confusion matrix (row-normalized).

Table VII
Cross-dataset generalization (CICIDS2017 $\leftrightarrow$ CSE-CIC-IDS2018, stratified sample seed=42; values from cross_dataset_results.csv).

Train	Test	Type	F1
CICIDS2017	CICIDS2017	in-domain	*[xd_aa]*
CICIDS2017	CSE-CIC-2018	cross-dataset	*[xd_ab]*
CSE-CIC-2018	CSE-CIC-2018	in-domain	*[xd_bb]*
CSE-CIC-2018	CICIDS2017	cross-dataset	*[xd_ba]*

V. Discussion

Method comparison. When the F1 gap between RF Top-30 and SHAP Top-30 is smaller than the variation across resamplings and $p > 0.05$, we conclude that the two methods are *statistically equivalent*; in that case secondary criteria (explainability, size, stability) decide the deployment choice. SHAP Top-30 provides consistent local explanations, an advantage for SOC operations.

Bridge to the edge device. Cutting $\approx$60% of the features directly reduces the vector-assembly cost and the model memory footprint on the Jetson Orin Nano Super. The selection criterion therefore needs to be *edge-aware*, that is, to consider not only F1 but also the number of features, the model size, and the training/prediction time (Figure 3). Latency and throughput measurements on the Jetson are presented in a companion systems paper (under submission).

Limitations of the binary evaluation. Removing the port and deduplicating brings binary F1 down to a realistic level, but binary classification remains a relatively easy task. Only under multiclass evaluation do the weaknesses on rare attack classes emerge, and this is also a more reliable metric for distinguishing methods.

VI. Threats to Validity

Internal validity. The RF/SHAP rankings are derived from the original training set and kept fixed across resamplings. This is consistent with the "a priori configuration" design, but may introduce slight leakage in the ranking part. Re-ranking per resampling would be more rigorous but costly (especially SHAP); we make this trade-off explicit. The selection of the Top-3 component models for the ensemble is performed on a separate *validation set* held out from the training set, so the test set does not participate in model selection; the trade-off is that the base models are trained on a smaller portion of the data after extracting the validation set. In addition, class balancing is applied uniformly to seven models (Section III-D) but *not* to MLP, because Spark ML does not support sample weights for this model; the MLP result should therefore be read with that caveat.

External validity. CICIDS2017 (2017) may not reflect current traffic; verification on newer datasets (CSE-CIC-IDS2018, CIC-IoT) is needed. The default random split shuffles time; a time-based split mode is supported for a more realistic evaluation. The cross-dataset generalization conclusion (Section IV-F) is based on a *representative stratified sample* of CSE-CIC-IDS2018 (due to edge resource limits), not the whole dataset; the sample size is reported explicitly, and scaling to the full dataset is the next direction. The model assumes an attack distribution approximating CICIDS2017 and has not been evaluated against deliberately evasive (adversarial evasion) traffic. Moreover, feature-level deduplication (Section III-B) itself changes the class distribution between benign and attack, which should be kept in mind when interpreting the results.

Statistical validity. The small sample size (N=6) gives the test low *power*: because the permutation test is two-sided, the smallest attainable p is only $2/2^N$, so N=6 just barely reaches the 0.05 threshold and can hardly detect small effects. Furthermore, the resamplings *overlap*, thus violating the independence assumption of the t distribution; therefore the Nadeau–Bengio corrected confidence interval is reported in parallel (Section III-F) and is the primary basis for conclusions, while the standard t interval is for reference only. A remaining limitation is that the RF/SHAP feature ranking is kept fixed across resamplings (no per-round re-ranking), creating a mild selection leakage favoring the two feature-selection branches; per-round re-ranking (feasible for RF) is the next reinforcement direction, together with increasing N with nested cross-validation.

VII. Conclusion

This paper compares four dimensionality-reduction strategies for IDS on Spark ML under a leakage-aware and statistically tested evaluation pipeline. The core contribution is not merely "which method is best" but also *how to evaluate* *reliably*: removing the leaking feature, deterministic feature-level deduplication, model selection on a validation set, exact-enumeration paired testing with Nadeau–Bengio correction, multiclass evaluation, and CICIDS2017↔CSE-CIC-IDS2018 cross-dataset generalization (Section IV-F). Future directions: scaling to the full CSE-CIC-IDS2018 and to newer datasets (RoEduNet, CIC-IoT), verifying the transferability of the SHAP Top-30 set itself between the two datasets, hyperparameter optimization, and real-time deployment on the edge.

Acknowledgment

This research was funded by the University of Information Technology, Vietnam National University Ho Chi Minh City. The work was carried out at the University of Transport and Communications (Ho Chi Minh City Campus) and the University of Information Technology, VNU-HCM, as part of a graduation thesis. The authors declare no conflict of interest.

References

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.

[2] M. Ring, S. Wunderlich, D. Grüdl, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[3] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.

[4] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and É. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Risks and Security of Internet and Systems (CRiSIS 2022), Revised Selected Papers*, pp. 18–33, Springer, 2023.

[5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] M. A. Ferrag, L. Maglaras, H. Janicke, J. Jiang, and L. Shu, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *Journal of Information Security and Applications*, vol. 50, p. 102419, 2020.

[8] K. Alrawashdeh and C. Purdy, "Toward an online anomaly intrusion detection system based on deep learning," in *Proc. 15th IEEE Int. Conf. on Machine Learning and Applications (ICMLA)*, pp. 195–200, 2016.

[9] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.

[10] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud'10)*, 2010.

[11] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar, "MLlib: Machine learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

[12] K. Baalaji *et al.*, "Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms," *Discover Applied Sciences*, vol. 7, p. 928, 2025.

[13] M. Z. Asghar *et al.*, "HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning," *Scientific Reports*, vol. 16, p. 2313, 2026.

[14] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[15] I. T. Jolliffe, *Principal Component Analysis*. Springer Series in Statistics, Springer, 2nd ed., 2002.

[16] M. Sarhan, S. Layeghy, and M. Portmann, "Evaluating standard feature sets towards increased generalisability and explainability of ml-based network intrusion detection," *Big Data Research*, vol. 30, p. 100359, 2022.

[17] J. Gama, I. Zliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[18] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[19] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, G. Montavon, W. Samek, M. Kloft, T. G. Dietterich, and K.-R. Müller, "A unifying review of deep and shallow anomaly detection," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 756–795, 2021.

[20] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Conditional variational autoencoder for prediction and feature recovery applied to intrusion detection in iot," *Sensors*, vol. 17, no. 9, p. 1967, 2017.

[21] T. Hasan, A. Hossain, M. Q. Ansari, and T. H. Syed, "Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning," 2025. arXiv:2501.15266.